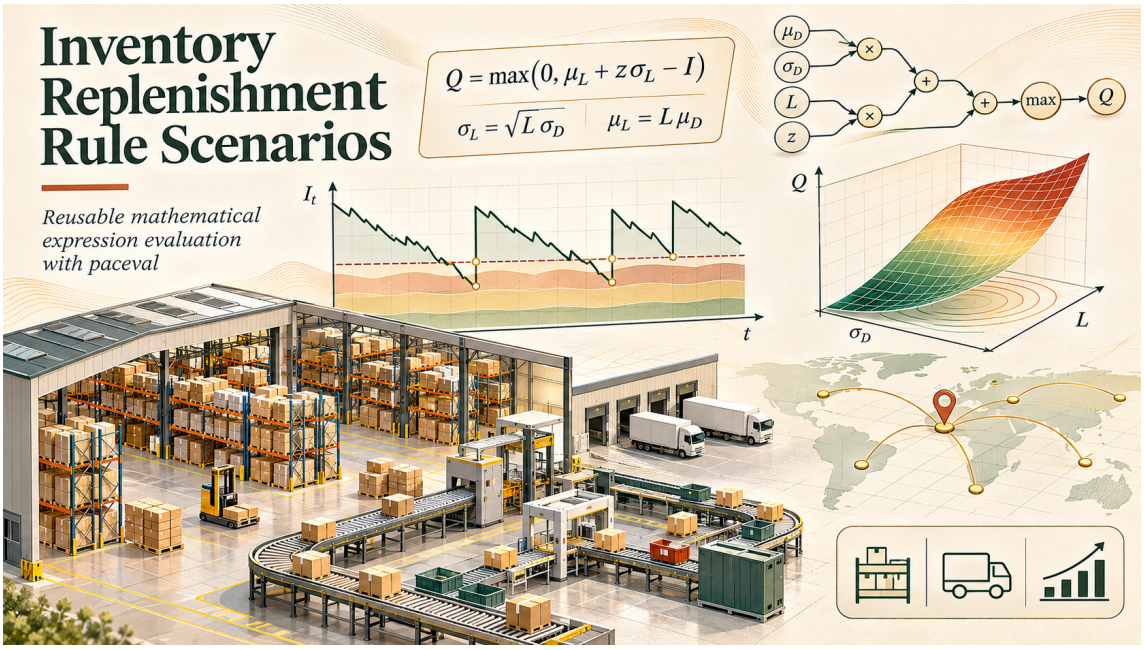
Inventory Replenishment Rule Scenarios

Versioned inventory-policy mathematics, direct reusable paceval computations, reproducible discrete-event scenarios, and independent numerical audits.



Cover illustration: concept artwork. All numbered charts are generated from the released paceval computation exports.

Observed outcome. Thirty registered reorder policies were implemented using 60 persistent paceval mathematical graphs. The released study evaluated 105,000 policy–SKU scenarios, independently audited 1,920 packed batch evaluations, and completed a 120-day synthetic inventory simulation for 1,440 policy–SKU combinations.

Scope. This is a controlled numerical feasibility pilot, not a production inventory recommendation, real supplier forecast, live ERP connection or statement of commercial benefit. No time or hardware-related performance comparison is included.

1. Industrial problem and execution boundary

Manufacturing, distribution and service-parts operations continuously revise reorder parameters as demand, lead times, review intervals and service objectives change. The software must evaluate numerous alternative rules while preserving the inventory-position, outstanding-order and demand-event semantics of the host system.

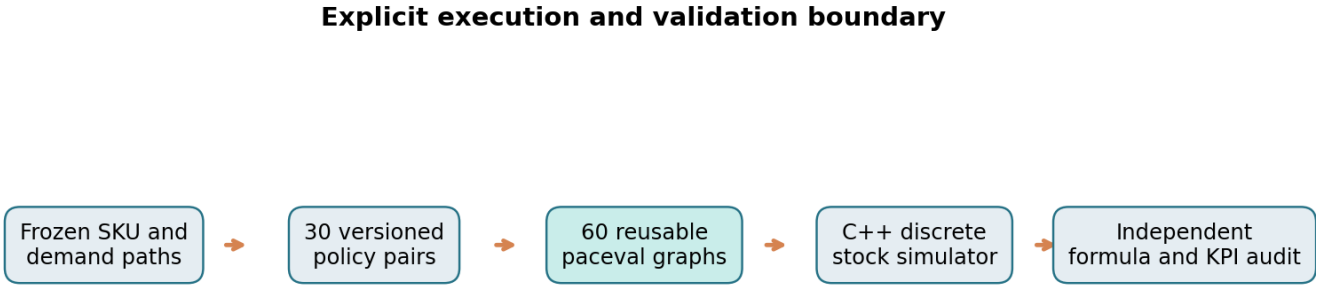


Figure 1. A fixed stateful host passes validated numeric inputs to versioned paceval computation handles, then audits results.

What paceval evaluates

The engine owns textual mathematical expressions, 60 persistent calculation handles, scalar double-precision calls and packed batch calls on independent inputs. Each registered policy contains an order-up-to target expression and a threshold expression.

What the C++ host evaluates

The host owns integer order rounding, policy gates, deterministic demand streams, fixed delivery schedules, stock arrivals, inventory position, lost-sales consumption, output files and separate native formula reference checks. Passing a mathematical evaluation does not establish that a replenishment policy is optimal.

2. Mathematical policy contracts

Definitions: mean daily demand m [units/day], standard deviation s [units/day], lead time L [days], review interval R [days], inventory position I [units], and demand trend h [units/day]. Policy-specific z, b and k are dimensionless experimental factors, except b denotes a dimensionless equivalent of buffer days multiplied by m. The formula assumes independent daily demand only for the square-root uncertainty proxy; no fitted demand distribution or probabilistic service guarantee is asserted.

Order-up-to gap: reusable paceval expression

G = m(L+R) + z s sqrt(L+R) + b m + k h L - I

The candidate order is Q = ceil(max(0,G)) units. Positive-part clipping and rounding are host operations. All additions have units of inventory units when L and R are treated in days.

Conditional threshold: independent paceval expression

H = m L + z s sqrt(L) + b m - I

Conditional policies place an order only if H > 0. Other policies compute the positive order-up-to quantity on their specified review day. Inventory position equals on-hand units plus outstanding orders, with no reserved/backordered units in the lost-sales study.

Discrete inventory dynamics

At each daily step t, the host receives scheduled arrivals A_t and observes demand D_t. Immediate fulfillment is F_t = min(I_t + A_t, D_t). Closing stock is max(0, I_t + A_t - D_t). New orders enter a future arrival bucket at t + L. These operations are deliberately not misrepresented as formulas executed by paceval.

3. Controlled test workload and reproducibility

The main released test contains 3,500 deterministically generated SKUs crossed with 30 versioned rules, totaling 105,000 static formula scenarios. Each policy has two separate graphs using a shared six-variable ABI; values are supplied without rebuilding the expression. Thirty-two inputs per graph are also checked through the packed SDK API.

SKU combinations	3,500 SKUs x 30 policies = 105,000 policy–SKU cases
Calculation graph handles	60: thirty order-up-to gaps and thirty conditional trigger gaps
Scalar formula audits	213,721, including the 61 x 61 formula surface
Packed batch audits	1,920 values: 32 inputs x 60 graphs
Inventory simulation	48 SKUs x 30 policies x 120 days = 1,440 synthetic runs
Demand construction	Frozen per-SKU deterministic pseudorandom sequence, seasonal multiplier, nonnegative demand
Review/lead conventions	Daily receiving and demand consumption; review only at declared integer intervals
No user data	Synthetic products, supplier lead times and demand values

Results are exported as human-readable CSV; a separate Python high-precision oracle recomputes saved expression samples, and CTest invokes the original SDK integration path.

4. Verified numerical execution

The original paceval_main.cpp and paceval_main.h public SDK facade is built into the C++ executable and linked to a compatible external Linux runtime. The measured result is correctness of mathematical evaluation in the stated bounded domain, not superiority to other implementations.

Scalar formula audits	213,721 calculations passed the separately coded native C++ reference
Batch formula audits	1,920 per-item results passed the native reference and SDK error checks
Independent decimal checks	3600 / 3600 exported formula values agree within tolerance
Largest decimal difference	3.506e-13 inventory units (CSV-decimal interpretation)
Inventory outcome agreement	1,440 / 1,440 paired runs had the same order counts and fill-rate outcomes
CMake / CTest	1 / 1 integration test passed with externally supplied paceval Linux .so
Reported runtime speedups	None: not a study objective in this release

The external decimal reference operates on the released CSV decimal values, not bit-exact reconstruction of all binary64 inputs. The numerical tolerance is abs(error) <= 1e-10 * (1 + abs(reference)). All released observations are finite and the independently regenerated results passed the coded acceptance tests. This finite sampling is not a proof for all parameter values.

5. Evaluated service and inventory outcomes

For this synthetic lost-sales simulation, fill rate means total immediate units fulfilled divided by total requested units. A cycle service level would instead measure the probability of a no-stockout replenishment cycle; it is not inferred from fill rate. No observed future demand is read when making an earlier review decision.

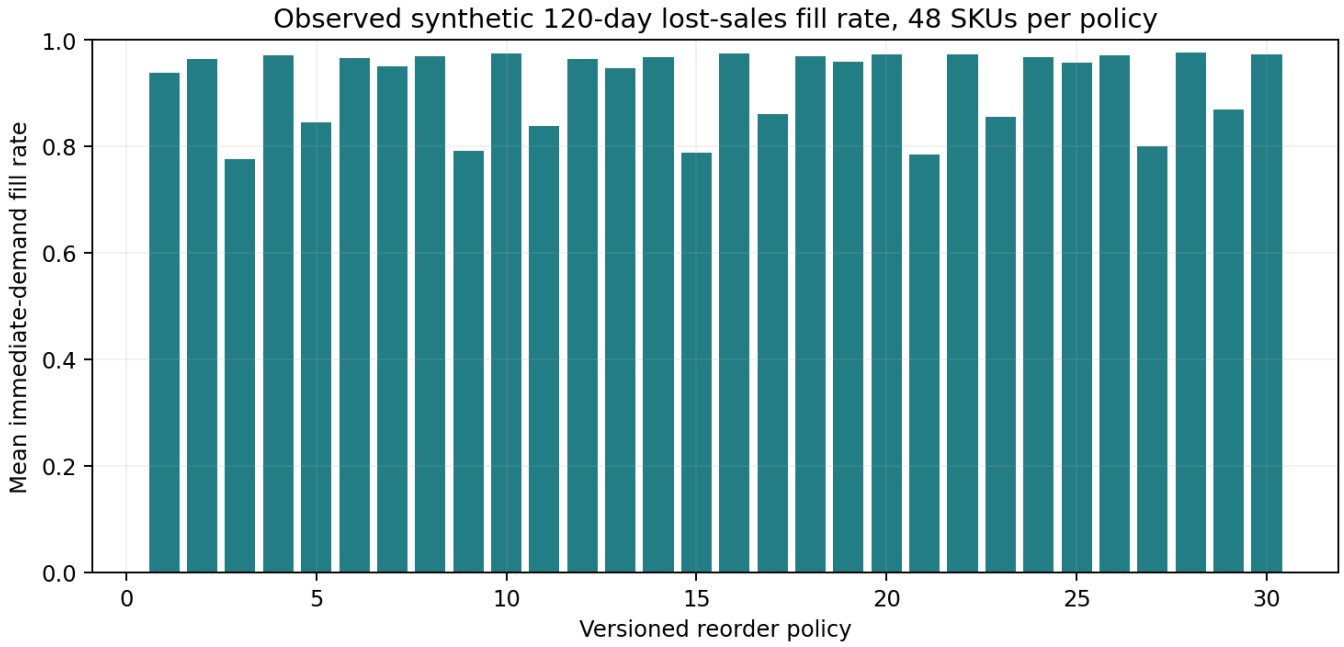


Figure 2. Different policy parameter combinations produce different realized fill-rate outcomes on identical SKU demand streams.

These measurements are simulation outputs from frozen, synthetic demand assumptions. They do not validate the underlying demand-generation model or guarantee a specified customer service level.

6. Reusable formula evaluation and safety rules

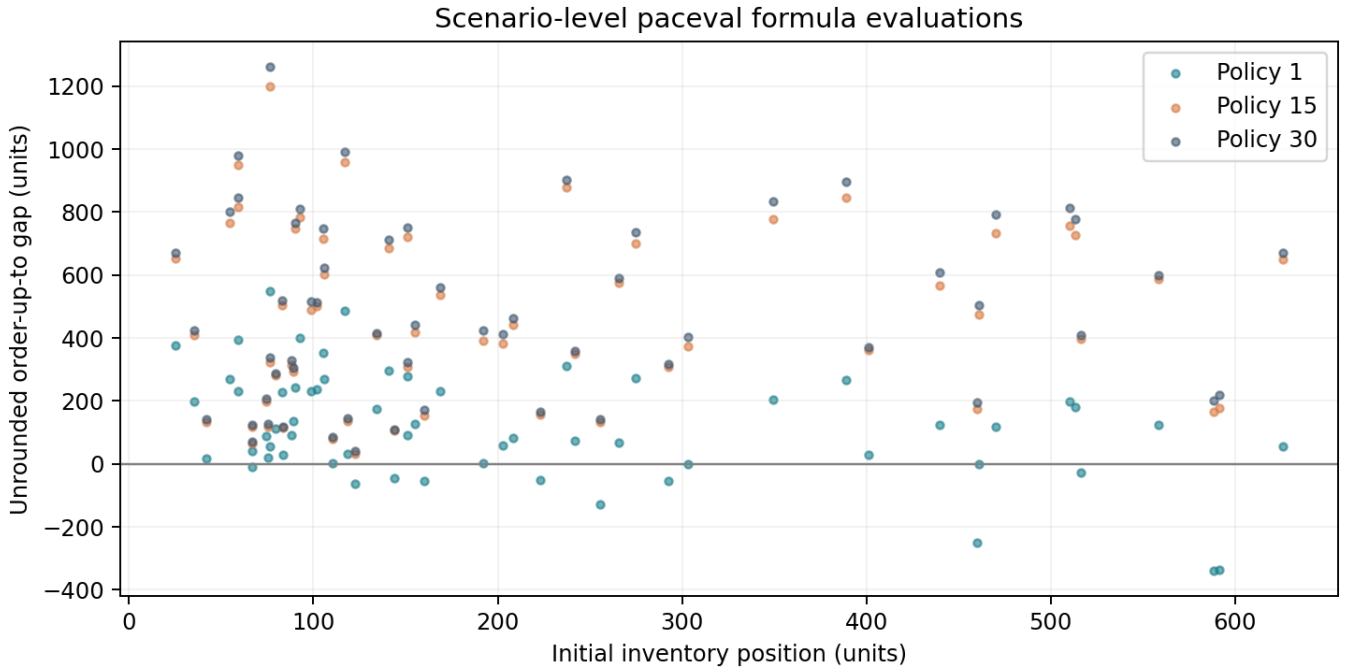


Figure 3. Evaluated unrounded order-gap values across selected policies and initial inventory positions.

The host preserves the declared formula revision and does not silently substitute a new target. A conditional gate is computed by a second persistent graph. Integer order quantities are rounded upward only after validating the floating-point formula result.

Important assumptions

No minimum order quantity, supplier calendar, bulk discounts, perishability, multi-echelon transfers, returns, stock reservations, substitutions, transport capacity or dynamically updated forecasts are modeled. The review dates do not imply continuous monitoring between checks.

7. Real 3D mathematical scenario surface

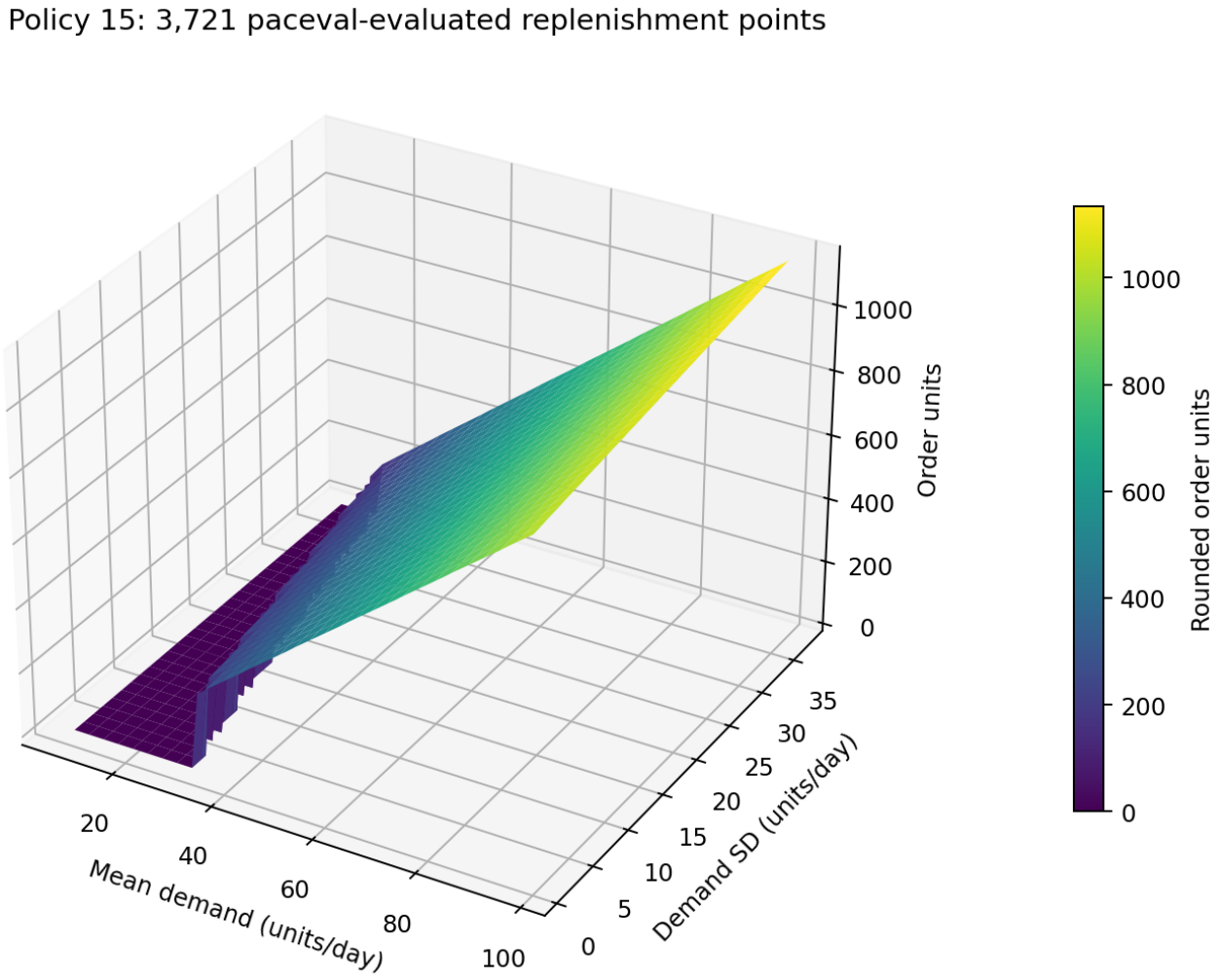


Figure 4. A 61 x 61 order-quantity surface from 3,721 direct paceval calls using one persistent graph.

The stair-step appearance follows from integer ceil and positive-part clipping performed by the host. The raw mathematical graph remains continuous in its inputs before rounding and policy gating.

8. Contours and service–inventory trade-offs

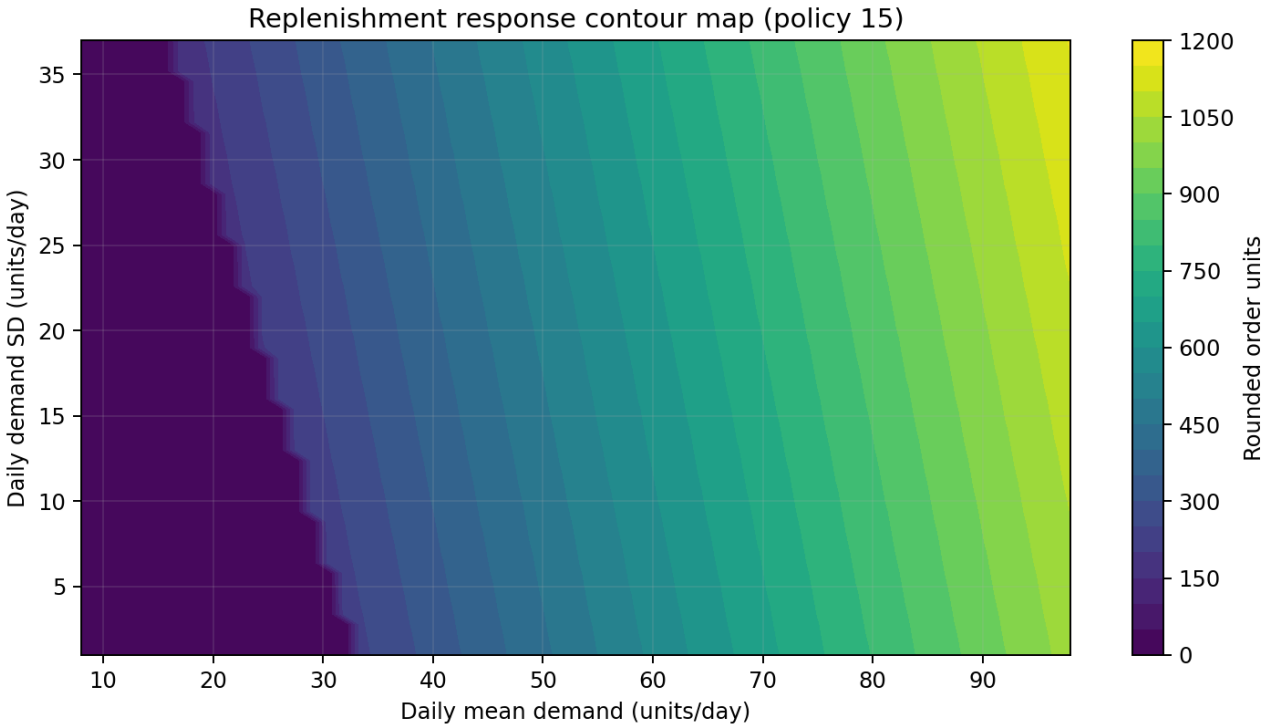


Figure 5. Contour projection of the same 3,721 computed order quantities.

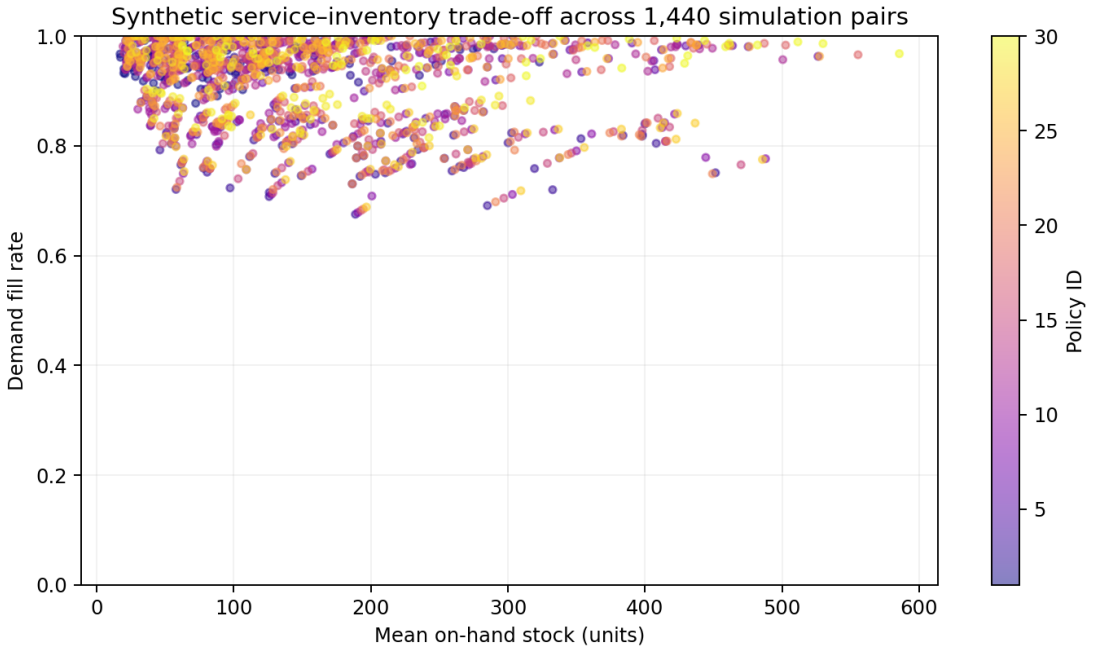


Figure 6. Realized synthetic fill rate versus mean stock for each of 1,440 simulated policy–SKU combinations.

9. Policy versioning and research interpretation

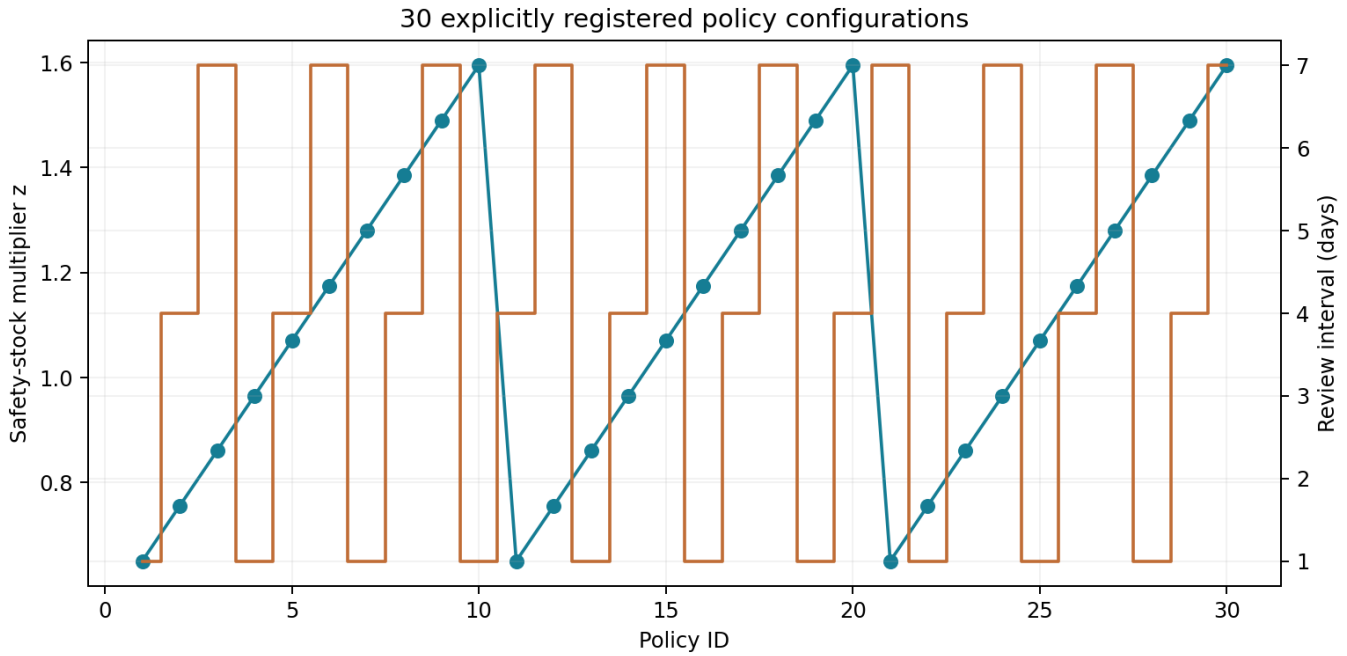


Figure 7. Fixed policy-specific parameters and review cadence; the complete registry is machine readable.

Thirty distinct policy configurations are registered as transparent formula strings; both the order-up-to and optional trigger calculations can be inspected and reproduced. Future revisions should register a new policy ID, retain the previous graph, and perform the same numerical audit before activation.

Scientific interpretation

The demonstration establishes that paceval can manage and evaluate reusable mathematical rule kernels across many scenarios while a C++ inventory simulation handles discrete state changes. It does not establish statistical optimality, production ERP compatibility or real customer service performance.

10. Reproduction, limitations and references

Build: cmake -S src -B build -DCMAKE_BUILD_TYPE=Release -DPACEVAL_RUNTIME=/absolute/path/to/libpaceval_linux_sharedLIB.so. Then compile with cmake --build build -j and run ctest --test-dir build --output-on-failure. The external binary is not included in the publication ZIP.

Recompute: run ./build/inventory_replenishment --skus 3500 --out reproduced; compare or replace the released examples explicitly before regenerating charts. Python scripts regenerate figures and this PDF from the files under examples/.

Scope and limitations

Service levels depend on demand, lead-time and lost-sales assumptions. The project does not include real inventory transactions, forecasting, policy optimization, optimization guarantees, supplier reliability models or probabilistic coverage certification. No performance or energy advantage is claimed.

External scientific and technical resources

1. paceval official mathematical API: https://paceval.com/api/

2. Inventory Systems, continuous and periodic review: https://rossetti.github.io/RossettiInventoryBook/08-Chapter8.html

3. Continuous versus periodic review discussion, TUM: https://mediatum.ub.tum.de/doc/1743194/1743194.pdf

4. Inventory service metrics, NIDA: https://logistics.nida.ac.th/wp-content/uploads/2016/08/6001-11-multiperiod-performance-v2.pdf

Conclusion: The defined inventory rule evaluations with 60 real paceval computation graphs were executed, cross-checked against separate formulas, and integrated with a deterministic 120-day lost-sales simulator. The evidence supports numerical feasibility in this controlled research domain.